

MOP & System Architecture Guide

MCH Surveillance System — উইন্ডোজ এনভায়রনমেন্টে নেটিভ PostgreSQL ব্যবহার করে ব্যাকএন্ড (IntelliJ IDEA) ও ফ্রন্টএন্ড (Android Studio) চালানোর ধাপে ধাপে নির্দেশনা, সিস্টেমের মৌলিক আর্কিটেকচার, ডেটা ফ্লো, যাচাই ও ট্রাবলশুটিং

Version 2.0 · 06 Jul 2026 · Branch: `claude/spring-boot-startup-error-d5igrb` · Repo: `shakil128/Preventive-Healthcare-App-Bangladesh`

১. সিস্টেম আর্কিটেকচার ও কম্পোনেন্ট	পৃষ্ঠা ১
২. এন্ড-টু-এন্ড ডেটা ফ্লো (৪টি প্রধান ফ্লো)	পৃষ্ঠা ২
৩. MOP পার্ট ১ — ডাটাবেজ ও ব্যাকএন্ড (IntelliJ IDEA)	পৃষ্ঠা ৩
৪. MOP পার্ট ২ — ফ্রন্টএন্ড (Android Studio)	পৃষ্ঠা ৪
৫. ব্যবহার-ফ্লো: Admin onboarding ও Field officer কাজ	পৃষ্ঠা ৪
৬. যাচাইকরণ (SQL + API smoke test)	পৃষ্ঠা ৫
৭. ট্রাবলশুটিং — সাধারণ এরর ও সমাধান	পৃষ্ঠা ৫
৮. Optimization Roadmap ও নিরাপত্তা নোট	পৃষ্ঠা ৬

⚠ পুরোনো গাইডের ৩টি ভুল — এই সংস্করণে সংশোধিত:

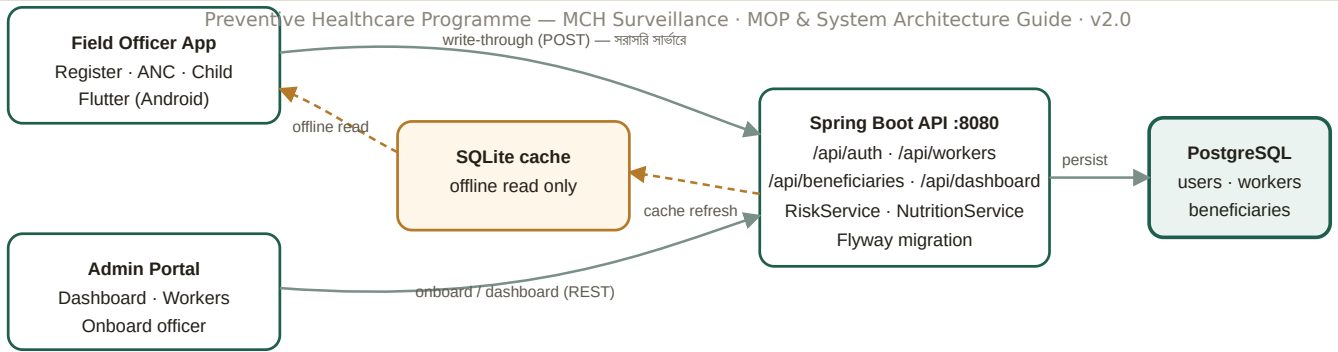
পুরোনো (ভুল) নির্দেশনা	সঠিক তথ্য (কোডবেস অনুযায়ী)
Main class: <code>MchSurveillanceBackendApplication</code>	✓ <code>bd.gov.mch.MchApplication</code>
"CRITICAL": <code>@Column</code> annotation নিজে যোগ করা	✓ দরকার নেই — annotation আগে থেকেই আছে; আবার দিলে duplicate → compile error
<code>flutter pub run build_runner build ...</code> চালানো	✓ চালাবেন না — প্রজেক্টে code-generator নেই, কমান্ড fail করবে
লোকাল ডাটাবেজ "Isar/Hive"	✓ আসলে sqflite (SQLite) — শুধুই offline cache
Base URL ফাইল "constants.dart"	✓ আসলে <code>mobile/lib/config/api_config.dart</code>

১ - সিস্টেম আর্কিটেকচার ও কম্পোনেন্ট

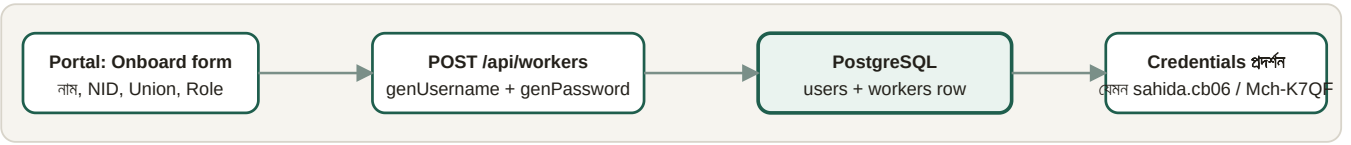
সিস্টেমটি **PostgreSQL-কেন্দ্রিক (single source of truth)** আর্কিটেকচার অনুসরণ করে: প্রতিটি লেখা (write) সরাসরি সার্ভারে যায় ও PostgreSQL-এ স্থায়ী হয়; প্রতিটি পড়া (read) অনলাইনে সার্ভার থেকে রিফ্রেশ হয়। ডিভাইসের লোকাল SQLite শুধু **offline cache** — ইন্টারনেট না থাকলে সর্বশেষ synced ডেটা দেখায়।

কম্পোনেন্ট	প্রযুক্তি	দায়িত্ব
Flutter মোবাইল অ্যাপ (Field App + Admin Portal)	Flutter 3.22+, provider, dio, sqflite	স্বাস্থ্যকর্মীর মাঠের কাজ (নিবন্ধন, ANC visit, child assessment) এবং সুপারভাইজারের পোর্টাল (dashboard, worker onboarding)। sqflite = শুধু read-cache।
Spring Boot API (Backend)	Java 21, Spring Boot 3.3.5, JPA + Flyway	সব business logic: login যাচাই, RiskService (ANC ঝুঁকি) ও NutritionService (MUAC/SAM/MAM) ক্লাসিফিকেশন, credential জেনারেশন, dashboard aggregation।
PostgreSQL (Database)	PostgreSQL (Windows-এ নেটিভ), schema Flyway-managed	সব দীর্ঘমেয়াদী ডেটা: <code>users</code> , <code>workers</code> , <code>beneficiaries</code> । Hibernate <code>ddl-auto: validate</code> — schema-র মালিক Flyway migration।

write-through (POST) — সরাসরি সার্ভারে



ফ্লো A — Field officer অ্যাকাউন্ট তৈরি (Admin onboarding)



Username/password সার্ভারে তৈরি হয় — admin যা দেখেন হুবহু তা-ই ডাটাবেজে থাকে, তাই লগইন কখনো mismatch হয় না। সার্ভারে না পৌঁছালে অ্যাপ স্পষ্ট এরর দেখায় — অসম্পূর্ণ "ভুলুডে" অ্যাকাউন্ট তৈরি হয় না।

ফ্লো B — নতুন মা/শিশু নিবন্ধন (Registration, write-through)



ফ্লো C — ক্লিনিক্যাল ভিজিট ও সার্ভার-সাইড ঝুঁকি নির্ণয়

ভিজিট	Endpoint	সার্ভারে কী হয়
ANC visit (গর্ভবতী মা)	POST /api/beneficiaries/{id}/anc-visit	RiskService: SBP $\geq$ 140 বা DBP $\geq$ 90, Hb (রক্তশুল্কতা), danger signs ইত্যাদি থেকে High/Medium/Low নির্ণয় → beneficiary আপডেট → PostgreSQL-এ সেভ → আপডেটেড রেকর্ড অ্যাপে ফেরত।
Child assessment (শিশু)	POST /api/beneficiaries/{id}/child-assessment	NutritionService: MUAC < 11.5 cm বা edema → SAM (High), 11.5-12.5 → MAM (Medium), নাহলে Normal → PostgreSQL-এ সেভ।
Live preview	POST /api/risk/anc , /api/nutrition/muac	ফর্মের টাইপ করার সাথে সাথে লাইভ ক্লাসিফিকেশন — persist করে না।

কেন সার্ভার-সাইড ক্লাসিফিকেশন? অ্যাপ ও পোর্টাল একই RiskService ব্যবহার করে, তাই একটি কেসের ঝুঁকি-স্ট্যাটাস নিয়ে দুই জায়গায় কখনো ভিন্ন ফল আসে না; ক্লিনিক্যাল নিয়ম বদলাতে হলে শুধু ব্যাকএন্ড আপডেট করলেই সব ক্লায়েন্ট নতুন নিয়ম পায়।

ফ্লো D — পড়া (Read-through) ও অফলাইন আচরণ

অবস্থা	আচরণ
অনলাইন	প্রতিটি তালিকা-লোডে অ্যাপ অ্যাপে GET /api/beneficiaries / GET /api/workers কল করে, সার্ভারের authoritative snapshot দিয়ে SQLite cache পুরো রিফ্রেশ করে, তারপর UI-তে দেখায়।
অফলাইন	সার্ভার call ব্যর্থ হলে সর্বশেষ cached snapshot থেকে পড়া হয় — মাঠকর্মী তালিকা দেখতে পান। লেখা (নতুন নিবন্ধন/ভিজিট) অফলাইনে ব্লক — অ্যাপ বাংলা+ইংরেজি এরর দেখায়, যাতে ডাটাবেজে না-পৌঁছানো এন্ট্রি "সেভ হয়েছে" বলে ভুল ধারণা না হয়। (Roadmap: durable offline write-queue, সেকশন ৮)

ফ্লো E — তদারকি (Supervision)

সুপারভাইজার পোর্টাল GET /api/dashboard ও GET /api/indicators কল করে PostgreSQL-এর লাইভ ডেটা থেকে কভারেজ, ঝুঁকি-বন্টন ও পুষ্টি পরিসংখ্যান দেখেন — field officer-দের সাম্প্রতিক এন্ট্রিগুলো সাথে সাথেই প্রতিক্রিয়া পায়।

পূর্বশর্ত: Java 21, Flutter SDK (৩.২২+), PostgreSQL, IntelliJ IDEA, Android Studio ইনস্টল থাকতে হবে। এবং সঠিক ব্রাঞ্চে থাকুন — সব PostgreSQL ফিক্স আছে `claude/spring-boot-startup-error-d5igrb` -এ:
`git fetch origin && git checkout claude/spring-boot-startup-error-d5igrb && git pull`

ধাপ ১.১ — PostgreSQL-এ ডাটাবেজ ও ইউজার তৈরি

pgAdmin অথবা psql টার্মিনালে (postgres superuser হিসেবে):

```
-- ইউজার না থাকলে তৈরি করুন
CREATE USER zahra WITH PASSWORD 'zahra';
-- ডাটাবেজ তৈরি করে zahra-কে Owner বানান (Flyway-র টেবিল CREATE করার অধিকার লাগে)
CREATE DATABASE preventive_healthcare OWNER zahra;
```

যাচাই: `psql -U zahra -d preventive_healthcare -h localhost` দিয়ে চোকা গেলে ✓

ধাপ ১.২ — প্রজেক্ট খোলা ও build রিফ্রেশ

#	কাজ
1	IntelliJ-এ <code>backend</code> ফোল্ডার খুলুন; Maven সব dependency নামানো পর্যন্ত অপেক্ষা করুন।
2	ডান পাশের Maven ট্যাব → Reload All Maven Projects (🔄)।
3	Build ▶ Rebuild Project — এতে <code>application.yml</code> ও <code>db/migration/V1__init.sql</code> <code>target/classes</code> -এ কপি হয়। এই ধাপ বাদ দিলে আপ random in-memory H2 -তে চলে ও "Table BENEFICIARIES not found" এরর আসে (ট্রাবলশুটিং T1)।

ধাপ ১.৩ — Run Configuration

সেটিং	মান
Configuration type	Spring Boot (বা Application)
Main class	<code>bd.gov.mch.MchApplication</code> (MchSurveillanceBackendApplication নয়!)
Active profiles / Env vars	খালি রাখুন — <code>application.yml</code> -এ PostgreSQL ডিফল্ট (<code>preventive_healthcare / zahra</code>) আপে থেকেই সেট। প্রয়োজনে <code>DB_URL</code> , <code>DB_USERNAME</code> , <code>DB_PASSWORD</code> env var দিয়ে override করা যায়।

এনটিটি কোড ছোঁবেন না: `Worker.union` → `union_name` , `User.role` → `user_role` , `Beneficiary.next` → `next_visit` , `day` → `pnc_day` — সব `@Column` ম্যাপিং সোর্সে আপে থেকেই আছে এবং `V1__init.sql` -এর সাথে হুবহু মিলে; `ddl-auto: validate` pass করবে। নতুন করে annotation দিলে duplicate-এ compile ভাঙবে।

ধাপ ১.৪ — Run ▶ ও সফলতার লক্ষণ

সবুজ Run বাটনে ক্লিক করুন। টার্মিনালে এই লাইনগুলো **ক্রমাগত** দেখতে পাবেন:

```
HikariPool-1 - Start completed.           ← jdbc:postgresql://localhost:5432/preventive_healthcare
Successfully applied 1 migration ... V1 - init ← Flyway টেবিল তৈরি করল (প্রথমবার)
Started MchApplication in X.X seconds       ← সার্ভার রেডি, port 8080
```

লগে `jdbc:h2:mem:...` দেখলে বুঝবেন কনফিগ `classpath`-এ যায়নি — ধাপ ১.২(৩) আবার করুন। আর যদি Flyway এরর বা `validate-fail` আসে, ট্রাবলশুটিং টেবিল (সেকশন ৭) দেখুন।

ধাপ ২.১ — প্রজেক্ট রেডি করা

```
flutter clean
flutter pub get
```

❌ **flutter pub run build_runner build ...** চালাবেন না — এই প্রজেক্টে কোনো code generator (freezed/json_serializable/Isar) নেই; কমান্ডটি "build_runner not found" এররে fail করবে। মডেলগুলো হাতে লেখা।

ধাপ ২.২ — API Base URL

টার্গেট	Base URL	কীভাবে সেট করবেন
Android Emulator	http://10.0.2.2:8080	কিছুই করতে হবে না — এটাই ডিফল্ট (mobile/lib/config/api_config.dart)
আসল ফোন (USB, একই WiFi)	http://<PC-LAN-IP>:8080	Run args-এ: --dart-define=API_BASE_URL=http://192.168.0.10:8080 (IP জানতে ipconfig)
Chrome (web)	http://localhost:8080	flutter run -d chrome --dart-define=API_BASE_URL=http://localhost:8080

ফাইল এডিট না করে --dart-define ব্যবহার করাই ভালো — কোড অপরিবর্তিত থাকে। (ফাইলের নাম api_config.dart ; পুরোনো গাইডের "constants.dart" নামে কোনো ফাইল নেই।)

ধাপ ২.৩ — অ্যাপ চালু

#	কাজ
1	Device Manager থেকে emulator চালু করুন (অথবা ফোন কানেক্ট করুন)।
2	Device dropdown-এ টার্গেট সিলেক্ট → সবুজ Run ► (অথবা টার্মিনালে flutter run)।
3	অ্যাপ চালু হলে এটি সরাসরি আপনার Windows-এর PostgreSQL-ব্যাংকড API-র সাথে যুক্ত।

৫ • ব্যবহার-ফ্লো: Admin ও Field Officer

৫.১ — Admin: field officer onboard

ধাপ	কাজ	নেপথ্যে
A1	Portal ভিউ → লগইন s.rahman / Admin-2026	POST /api/auth/login → users টেবিল যাচাই
A2	Sidebar ► Workers ► Onboard field officer — নাম, বয়স, মোবাইল, NID, Union, Role দিন	—
A3	Create account & generate login চাপুন	POST /api/workers → PostgreSQL-এ User+Worker
A4	প্রদর্শিত username/password Copy করে officer-কে নিরাপদে দিন	Credentials সার্ভার-জেনারেটেড — DB-র সাথে হুবহু মিল

৫.২ — Field officer: দৈনন্দিন কাজ

ধাপ	কাজ	নেপথ্যে
F1	Field ভিউতে A4-এর credentials দিয়ে লগইন	POST /api/auth/login

F2	New registration — ফর্ম পূরণ → Save & start first ANC visit	<code>POST /api/beneficiaries</code> → Postgres row
Preventive Healthcare Programme — MCH Surveillance · MOP & System Architecture Guide · v2.0		
F3	ANC ভিজিটে vitals (BP, Hb, danger signs) দিয়ে Save	সার্ভার risk classify → Postgres update
F4	শিশুর ক্ষেত্রে MUAC/edema দিয়ে assessment Save	সার্ভার SAM/MAM নির্ণয় → Postgres update

ডেমো field অ্যাকাউন্ট: `rokeya.cb01` / `Mch-2026`

৬ • যাচাইকরণ — PostgreSQL footprint ও API smoke test

Preventive Healthcare Programme — MCH Surveillance MOU & System Architecture Guide • v2.0

৬.১ — SQL যাচাই (psql বা IntelliJ Database টুল)

```
-- লগইন অ্যাকাউন্টসমূহ (নতুন officer-এর row থাকবে, role='field-officer')
SELECT username, user_role, name FROM users ORDER BY username;

-- মাঠকর্মী রেজিস্টার (নতুন onboard হওয়া worker সবার ওপরে)
SELECT id, name, role, union_name FROM workers ORDER BY id DESC;

-- মা/শিশু রেজিস্টার + সর্বশেষ ঝুঁকি-স্ট্যাটাস
SELECT id, name, type, risk, reason, muac FROM beneficiaries ORDER BY id DESC;
```

৬.২ — API smoke test (PowerShell/curl, backend চালু অবস্থায়)

```
# তালিকা আসছে কিনা
curl http://localhost:8080/api/beneficiaries
curl http://localhost:8080/api/workers

# লগইন যাচাই (200 = সফল, 401 = ভুল credentials)
curl -X POST http://localhost:8080/api/auth/login -H "Content-Type: application/json" ^
-d "{\"username\":\"s.rahman\", \"password\":\"Admin-2026\"}"
```

তিনটি টেবিলেই এন্ট্রি + API ২০০ রেসপন্স = **read ও write দুটোই PostgreSQL থেকে নিশ্চিত ✓**

৭ • ট্রাবলশুটিং — সাধারণ এরর ও সমাধান

#	লক্ষণ (লগ/এরর)	কারণ ও সমাধান
T1	Table "BENEFICIARIES" not found + লগে jdbc:h2:mem:...	target/classes -এ application.yml / db/migration কপি হয়নি → Maven Reload + Build ► Rebuild Project , তারপর target\classes\application.yml আছে কিনা দেখুন।
T2	Connection refused / password authentication failed	PostgreSQL সার্ভিস বন্ধ (services.msc-তে চালু করুন) বা ইউজার/পাসওয়ার্ড ভুল → ধাপ ১.১-এর SQL আবার চালান।
T3	Schema-validation: missing table/column	DB-তে পুরনো/অসম্পূর্ণ schema → খালি ডাটাবেজ নিয়ে আবার চালান (Flyway নিজে টেবিল বানাবে): DROP DATABASE preventive_healthcare; CREATE DATABASE preventive_healthcare OWNER zahra;
T4	Cannot find main class MchSurveillanceBackendApplication	ভুল ক্লাসনাম — Run Config-এ bd.gov.mch.MchApplication দিন।
T5	অ্যাপে "Could not reach the server / সার্ভারে সংযোগ ব্যর্থ"	Backend বন্ধ, বা ভুল Base URL। Emulator-এ 10.0.2.2:8080 ; আসল ফোনে PC-র LAN IP + একই WiFi + Windows Firewall-এ port 8080 allow।
T6	flutter pub run build_runner ... → "not found"	স্বাভাবিক — প্রজেক্টে build_runner নেই; কমান্ডটি বাদ দিন।
T7	Portal-এ পুরনো/ডেমো ডেটা দেখাচ্ছে, নতুন এন্ট্রি নেই	অ্যাপ অফলাইনে cache থেকে পড়ছে → নেটওয়ার্ক/Backend ঠিক করে তালিকা রিফ্রেশ করুন; অনলাইনে প্রতিটি লোড সার্ভার থেকে আসে।
T8	Flyway: Migration checksum mismatch	V1__init.sql এডিট করা হয়েছে → এডিট undo করুন; schema বদলাতে হলে নতুন V2__*.sql লিখুন (পুরনো migration কখনো এডিট নয়)।

৮.১ — এখনকার ডিজাইনে যা নিশ্চিত

- **SoT** সব read/write PostgreSQL-কেন্দ্রিক — এক সত্য উৎস, রিপোর্টিং সবসময় সঠিক।
- **Consistency** Credentials ও ব্লক-স্ট্যাটাস সার্ভার-জেনারেটেড — অ্যাপ/পোর্টাল/DB কখনো অমিল হয় না।
- **Safety** সার্ভারে না-পৌঁছানো লেখা ব্যর্থ হিসেবে স্পষ্ট দেখানো হয় — নীরব ডেটা-হারানো নেই।
- **Offline read** সংযোগ গেলেও মাঠকর্মী সর্বশেষ synced তালিকা দেখতে পান।

৮.২ — পরবর্তী optimization (অগ্রাধিকারক্রমে)

#	উন্নতি	বিবরণ ও লাভ
1	Durable offline write-queue	অফলাইনে নিবন্ধন/ভিজিট full payload-সহ লোকাল কিউতে রাখা, অনলাইন হলে আসল endpoint-এ replay (বর্তমান /api/sync শুধু ack করে)। প্রত্যন্ত অঞ্চলে অফলাইন লেখাও সম্ভব হবে।
2	Password hashing + টোকেন	BCrypt hash, প্রথম লগইনে বাধ্যতামূলক password বদল, JWT session। প্রোডাকশনের পূর্বশর্ত।
3	Delta sync	updated_at /ETag ভিত্তিক — পুরো তালিকার বদলে শুধু পরিবর্তিত রেকর্ড টানা; মোবাইল ডেটা ও লোড-টাইম কমে।
4	Pagination + server-side filter	Union/ব্লক অনুযায়ী সার্ভারে ফিল্টার — হাজারো beneficiary-তেও পোর্টাল দ্রুত।
5	Optimistic UI + retry/backoff	রেসপন্সের আগেই সাময়িক প্রদর্শন, ব্যর্থেরে rollback; নেটওয়ার্ক flaky হলে স্বয়ংক্রিয় পুনঃচেষ্টা।
6	Background sync worker	অ্যাপ বন্ধ থাকলেও নির্দিষ্ট বিরতিতে কিউ flush (WorkManager)।

৮.৩ — নিরাপত্তা নোট

- ডেমো credentials (s.rahman/Admin-2026 ইত্যাদি) ও plain-text password শুধু ডেমো বিস্তার জন্য — প্রোডাকশনে hash + টোকেন বাধ্যতামূলক।
- application.yml -এর DB পাসওয়ার্ড লোকাল dev-এর জন্য; আসল সার্ভারে DB_PASSWORD env var ব্যবহার করুন।
- CORS বর্তমানে * — প্রোডাকশনে নির্দিষ্ট origin-এ সীমিত করুন (mch.cors.allowed-origins)।

৮.৪ — সংশ্লিষ্ট ফাইল-ম্যাপ (রেফারেন্স)

ফাইল	ভূমিকা
backend/src/main/resources/application.yml	Datasource (PostgreSQL ডিফল্ট), Flyway, H2 ডেমো প্রোফাইল
backend/.../db/migration/V1__init.sql	Schema-র মালিক — users/workers/beneficiaries
backend/.../web/*Controller.java	Auth, Workers (onboard), Beneficiaries, Visits, Dashboard, Sync
backend/.../service/RiskService, NutritionService	ক্লিনিক্যাল ক্লাসিফিকেশন (server-side)
mobile/lib/config/api_config.dart	Base URL (ডিফল্ট 10.0.2.2:8080 , --dart-define override)
mobile/lib/services/api_service.dart	সব REST call (dio)
mobile/lib/services/repository.dart	Read-through + write-through অর্কেস্ট্রেশন
mobile/lib/services/local_db.dart	SQLite cache (sqflite)
mobile/lib/pages/RegisterPage, AncVisitPage, ChildHealthPage, OnboardModal	UI ফর্ম → সার্ভারে persist